

Companion Document: `fit_modular.R`

Full CAVI Loop — Supervised Matrix Factorization

Andrew Walther

2026-03-25

Table of contents

1	Overview	2
1.1	Relationship to Other Companion Documents	2
1.2	Relationship to <code>results/run_modular_simulation.R</code>	2
2	Mathematical Background	2
2.1	Model	2
2.2	Mean-Field Variational Posterior	3
2.3	Cox Taylor Expansion	3
2.4	Convergence Criterion	4
3	Algorithm Walkthrough	4
3.1	Initialization	4
3.2	Main CAVI Loop	5
3.2.1	Step 1 — Cox Taylor Expansion	5
3.2.2	Step 2 — Factor-Wise Inner Loop ($k = 1, \dots, K$)	5
3.2.3	Step 3 — Noise Precision Update	6
3.2.4	Step 4 — Convergence Check	6
4	Function Reference	7
4.1	Arguments	7
4.2	Return Value	7
5	<code>DATA_MODE</code> Toggle	8
6	Distinction from <code>results/run_modular_simulation.R</code>	9
6.1	Why Factor-Wise Beats Blockwise	9
7	Related Files	10

1 Overview

`code/fit_modular.R` wraps the four modular update functions into a single reusable fitting function, `fit_supervised_mf_modular()`. It implements **V3 Algorithm 1**: factor-wise Gauss-Seidel CAVI, updating $q(l_k) \rightarrow q(f_k) \rightarrow q(\beta_k)$ for each factor $k = 1, \dots, K$ within each outer iteration, then updating $q(\tau)$ once at the end.

1.1 Relationship to Other Companion Documents

This document sits at the top of the modular companion hierarchy. The four update modules each have their own self-contained companion documents that describe the mathematics, function signatures, tests, and demos for each individual coordinate update:

- `docs/update_beta.qmd` — $q(\cdot)$: scalar EBNM for Cox survival coefficients
- `docs/update_L.qmd` — $q(L)$: vector EBNM for patient loadings, dual-source (genomics + survival)
- `docs/update_F.qmd` — $q(F)$: vector EBNM for factor weights, pure genomics
- `docs/update_tau.qmd` — $q(\cdot)$: closed-form MLE for noise precision

`fit_modular.R` orchestrates these four modules. It does not re-implement their mathematics; it calls the exported functions from each module in the correct order, manages state (the posterior moment matrices `EL`, `EL2`, `EF`, `EF2`, `EBeta`, `EBeta2`, `Tau`), and handles initialization, the Cox Taylor linearisation, and convergence checking.

1.2 Relationship to `results/run_modular_simulation.R`

`results/run_modular_simulation.R` is an older benchmark script that uses the blockwise `_all` variants (`update_L_all`, `update_F_all`, `update_beta_all`, `update_tau`) and was written to benchmark simulation performance and produce figures and tables in `results/`. It is considered a deprecated approach: the blockwise update order (all-L, then all-F, then all-) is less canonical than the factor-wise Gauss-Seidel order specified in V3 Algorithm 1.

`fit_modular.R` is the **canonical, reusable fitting function**. It returns a list of posterior moments and is designed to be called on real data or simulation data interchangeably via the `DATA_MODE` toggle.

2 Mathematical Background

2.1 Model

The joint model links genomics observations to a low-rank latent factor structure, while simultaneously regressing patient survival on the same latent scores:

$$Y = LF^\top + E \quad (\text{genomics observation model})$$

$$h(t_i | l_i) = h_0(t_i) \exp\left(\sum_{k=1}^K l_{ik} \beta_k\right) \quad (\text{Cox PH survival model})$$

where:

- Y is the $n \times p$ genomics data matrix
- L is the $n \times K$ patient loading matrix (latent factor scores)
- F is the $p \times K$ feature weight matrix (factor loadings on genes)
- β is a K -vector of survival coefficients
- $E_{ij} \sim \mathcal{N}(0, \tau_j^{-1})$ is Gaussian noise with feature-specific precision τ_j
- $h_0(t)$ is an unspecified baseline hazard

The parameters L , F , β , and τ are all treated as latent quantities with variational posteriors.

2.2 Mean-Field Variational Posterior

The full posterior $p(L, F, \beta, \tau | Y, t, \delta)$ is intractable. CAVI approximates it with the mean-field factorization:

$$q(L, F, \beta, \tau) = q(L) q(F) q(\beta) q(\tau)$$

Within $q(L)$ and $q(F)$, each column is further factored over individuals and genes respectively. The ELBO objective is:

$$\mathcal{L} = \mathbb{E}_q[\log p(Y, t, \delta, L, F, \beta, \tau)] - \mathbb{E}_q[\log q(L, F, \beta, \tau)]$$

Each coordinate update maximises $\mathcal{L}$ with respect to one variational factor while holding the others fixed.

2.3 Cox Taylor Expansion

The Cox partial likelihood is non-conjugate and cannot be updated in closed form. CAVI linearises it via a second-order Taylor expansion around the current linear predictor:

$$\hat{\eta}_i = \sum_{k=1}^K \bar{l}_{ik} \bar{\beta}_k$$

The score (first derivative) and negative diagonal Hessian (curvature) at $\hat{\eta}$ are:

$$u_i = \left. \frac{\partial \ell_{\text{Cox}}}{\partial \eta_i} \right|_{\hat{\eta}}, \quad W_{ii} = - \left. \frac{\partial^2 \ell_{\text{Cox}}}{\partial \eta_i^2} \right|_{\hat{\eta}}$$

The second-order expansion gives a locally Gaussian form:

$$\ell_{\text{Cox}}(\eta) \approx \sum_i \left[-\frac{1}{2} W_{ii} (\eta_i - z_i)^2 \right] + C$$

where the **working response** is:

$$z_i = \hat{\eta}_i + \frac{u_i}{W_{ii}}$$

The function `calc_cox_taylor(eta, time, status)` computes u and w (the vector of W_{ii} values). The working response z is then formed in the main CAVI loop as `z <- eta + taylor$u / taylor$w`. The pair (z, w) is held fixed across all k within a single outer iteration.

2.4 Convergence Criterion

Convergence is declared when both of the following mean absolute changes fall below ε :

$$\Delta_L = \frac{1}{nK} \sum_{i,k} |\bar{l}_{ik}^{\text{new}} - \bar{l}_{ik}^{\text{old}}|, \quad \Delta_\beta = \frac{1}{K} \sum_k |\bar{\beta}_k^{\text{new}} - \bar{\beta}_k^{\text{old}}|$$

A 5-iteration burn-in is enforced before checking: the algorithm always runs at least 5 iterations.

Note on V3 Algorithm 1 specification: V3 Algorithm 1 nominally specifies the **maximum** absolute change as the convergence criterion. In practice, `fit_modular.R` uses the **mean** instead, because the max rarely falls below 10^{-5} on typical datasets. A small number of high-variance EL entries oscillate near factor orientation boundaries (SVD sign ambiguity), inflating the max without reflecting meaningful non-convergence in the bulk of the parameters. Using the mean with `tol = 1e-3` gives reliable, practical convergence.

3 Algorithm Walkthrough

This section traces through V3 Algorithm 1 as implemented in `fit_supervised_mf_modular()`.

3.1 Initialization

Before the CAVI loop begins, the algorithm initializes all posterior moments from the data:

SVD initialization of $\mathbf{L}$ and $\mathbf{F}$. A rank- K SVD of Y gives the starting subspace: $Y \approx U_K D_K V_K^\top$. Setting

$$\bar{L}^{(0)} = U_K D_K^{1/2}, \quad \bar{F}^{(0)} = V_K D_K^{1/2}$$

ensures $\bar{L}^{(0)}(\bar{F}^{(0)})^\top \approx Y_{\text{rank-}K}$ and places the starting point in the highest-variance subspace of the data. Second moments are initialized to squared means: $\overline{L^2}^{(0)} = (\bar{L}^{(0)})^2$ (zero initial posterior variance, updated at the first EBNM call).

Warm-start of β . Cox PH regression of survival on the SVD loadings $\bar{L}^{(0)}$ gives a warm start for $\bar{\beta}^{(0)}$, with $\overline{\beta^2}^{(0)} = (\bar{\beta}^{(0)})^2$.

Noise precision initialization. Each $\tau_j^{(0)}$ is set to the reciprocal of the sample variance of column j of Y , with a floor of 10^{-8} to avoid division by zero.

3.2 Main CAVI Loop

Each outer iteration $b = 1, 2, \dots$ proceeds through four steps.

3.2.1 Step 1 — Cox Taylor Expansion

Compute the current linear predictor:

$$\hat{\eta}_i^{(b)} = \sum_k \bar{l}_{ik}^{(b-1)} \bar{\beta}_k^{(b-1)}$$

Call `calc_cox_taylor(eta, time, status)` to obtain score u and weights w . Form the working response:

$$z_i^{(b)} = \hat{\eta}_i^{(b)} + \frac{u_i}{W_{ii}}$$

The pair $(z^{(b)}, w^{(b)})$ is computed **once per outer iteration** and held fixed across all k in Step 2.

3.2.2 Step 2 — Factor-Wise Inner Loop ($k = 1, \dots, K$)

For each factor k , perform three coordinate updates in sequence:

(a) Compute residual and partial working response. The residual excluding factor k is:

$$R^{(-k)} = Y - \bar{L}\bar{F}^\top + \bar{l}_k \bar{f}_k^\top$$

computed by `compute_R_k(Y, EL, EF, k)`. The partial working response excluding factor k is:

$$z_i^{(-k)} = z_i - \sum_{k' \neq k} \bar{l}_{ik'} \bar{\beta}_{k'}$$

computed by `compute_z_no_k(z, EL, EBeta, k)`. **Important:** $z^{(-k)}$ does not depend on $\bar{l}_{ik}$ or $\bar{\beta}_k$ — those terms cancel in the formula. It is therefore identical before and after the $q(l_k)$ update, and is safe to reuse for Step 2(f) without recomputing.

(b) Update $q(l_k)$ — Patient Loadings. Call `update_L_k(Tau, EF[,k], EF2[,k], w, EBeta[k], EBeta2[k], R_k, z_no_k)`.

This is a vector EBNM problem drawing signal from **two sources**: the genomics residual $R^{(-k)}$ and the survival working response $z^{(-k)}$. Write the results into `EL[,k]` and `EL2[,k]` immediately (Gauss-Seidel).

(c) Recompute $R^{(-k)}$ with updated $\bar{l}_k$. Call `compute_R_k(Y, EL, EF, k)` again, this time using the just-updated `EL[,k]`. This is the Gauss-Seidel step: $q(f_k)$ immediately benefits from the fresh $q(l_k)$.

(d) Update $q(f_k)$ — Factor Weights. Call `update_F_k(Tau, EL[,k], EL2[,k], R_k)`.

This is a vector EBNM problem using only the genomics signal. τ_j cancels in the pseudo-observation x_j (so the shrinkage level is τ -independent) but not in the noise level s_j (so features with high precision have tighter pseudo-likelihoods). Write the results into `EF[,k]` and `EF2[,k]`.

(e) Update $q(\beta_k)$ — Survival Coefficient. Call `update_beta_k(w, z_no_k, EL[,k], EL2[,k])`, reusing the $z^{(-k)}$ computed in step (a).

This is a scalar EBNM problem: a single pseudo-observation $x_k = B_k/A_k$ where

$$A_k = \sum_i w_i \mathbb{E}[l_{ik}^2], \quad B_k = \sum_i w_i z_i^{(-k)} \bar{l}_{ik}$$

Note that `EL[,k]` and `EL2[,k]` now reflect the updated values from step (b), so A_k incorporates the freshest second moments. Write the results into `EBeta[k]` and `EBeta2[k]`.

3.2.3 Step 3 — Noise Precision Update

After all K factors have been updated, call `update_tau(Y, EL, EL2, EF, EF2)`. This performs a closed-form MLE update (no EBNM) for each feature-specific precision τ_j :

$$\hat{\tau}_j = \frac{n}{\sum_i [(Y_{ij} - \bar{l}_i \bar{f}_j)^2 + \text{Var}_j]}$$

where Var_j is the variance correction term accounting for posterior uncertainty in L and F . The ELBO proxy is also computed here and stored in `history$elbo_proxy`.

3.2.4 Step 4 — Convergence Check

After iteration $b > 5$, compute:

$$\Delta_L = \text{mean}_{i,k} |\bar{l}_{ik}^{(b)} - \bar{l}_{ik}^{(b-1)}|, \quad \Delta_\beta = \text{mean}_k |\bar{\beta}_k^{(b)} - \bar{\beta}_k^{(b-1)}|$$

Stop if both $\Delta_L < \varepsilon$ and $\Delta_\beta < \varepsilon$.

4 Function Reference

```
fit_supervised_mf_modular(  
  Y,                # n x p genomics matrix  
  time,             # n-vector of survival/censoring times  
  status,           # n-vector of event indicators (1=event, 0=censored)  
  K = 5,            # number of latent factors  
  max_iter = 100,   # maximum CAVI iterations  
  tol = 1e-3,       # convergence threshold (mean absolute change)  
  verbose = TRUE    # print iteration summaries  
)
```

4.1 Arguments

Argument	Type	Default	Description
Y	$n \times p$ matrix	—	Genomics data matrix (samples $\times$ features)
time	n-vector	—	Observed survival or censoring times
status	n-vector	—	Event indicators: 1 = event occurred, 0 = censored
K	integer	5	Number of latent factors
max_iter	integer	100	Maximum outer CAVI iterations before stopping
tol	scalar	1e-3	Convergence threshold; both Δ_L and Δ_β must be below this value
verbose	logical	TRUE	If TRUE, prints iteration summaries every 10 iterations and a final convergence message

4.2 Return Value

The function returns a named list:

Field	Dimensions	Description
EL	$n \times K$	Posterior means $\mathbb{E}_q[l_{ik}]$ of patient loadings

Field	Dimensions	Description
EL2	$n \times K$	Posterior second moments $\mathbb{E}_q[l_{ik}^2] = \text{Var}_q(l_{ik}) + \bar{l}_{ik}^2$
EF	$p \times K$	Posterior means $\mathbb{E}_q[f_{jk}]$ of feature weights
EF2	$p \times K$	Posterior second moments $\mathbb{E}_q[f_{jk}^2]$
EBeta	K-vector	Posterior means $\mathbb{E}_q[\beta_k]$ of survival coefficients
EBeta2	K-vector	Posterior second moments $\mathbb{E}_q[\beta_k^2]$
Tau	p-vector	Feature-specific noise precisions $\hat{\tau}_j$
history	list	Convergence diagnostics (see below)

The `history` list contains:

Field	Description
<code>rmse</code>	Vector of reconstruction RMSE values, one per completed iteration
<code>elbo_proxy</code>	Vector of ELBO proxy values from <code>update_tau</code> , one per completed iteration
<code>converged</code>	Logical: TRUE if the algorithm converged before <code>max_iter</code>
<code>n_iter</code>	Number of outer iterations completed

E-prefix naming convention. The return fields use EL, EF, EBeta (not L, F, Beta) to signal that these are posterior expectations, not fixed parameter values. This convention matches the internal naming of the four modular update functions (`update_L_k` returns `$mean`, `update_beta_k` returns `$mean`, etc.) and avoids collision with base R names (F is FALSE in R; T is TRUE).

5 DATA_MODE Toggle

`fit_modular.R` supports two operating modes controlled by the `DATA_MODE` variable at the top of the file:

"real" (default). The file can be `source()`d without any side effects — no data is loaded and no fitting is performed. To fit on real data, set the three variables at the top of the file before running:


```
DATA_MODE    <- "real"
real_Y       <- your_n_by_p_matrix
real_time    <- your_survival_time_vector
real_status  <- your_event_indicator_vector
```

Then run:

```
Rscript code/fit_modular.R
```

"simulated". Running with `DATA_MODE <- "simulated"` activates the built-in DGP validation demo: $n = 250$, $p = 1000$, $K = 5$, seed 42, with $\beta_{\text{true}} = (1.5, -1.2, 0.8, -0.5, 0.0)$ and Weibull survival times. The same DGP parameters are used in `results/run_modular_simulation.R`, enabling direct comparison of the two fitting approaches.

6 Distinction from `results/run_modular_simulation.R`

The table below contrasts the two scripts:

Aspect	<code>run_modular_simulation.R</code>	<code>fit_modular.R</code>
Update order	Blockwise (<code>_all</code> variants): all-L $\rightarrow$ all-F $\rightarrow$ all- $\rightarrow$	Factor-wise (<code>_k</code> variants): for k : $L_k \rightarrow F_k \rightarrow \beta_k$
Convergence	Mean absolute change	Mean absolute change (V3 Algorithm 1 specifies max; mean used for practical convergence)
Purpose	Simulation benchmark with plots and tables	Reusable inference function
Output	Figures and tables written to <code>results/</code>	Return value (list of posterior moments)
Data	Hardcoded simulation DGP	<code>DATA_MODE</code> toggle (simulated or real)

6.1 Why Factor-Wise Beats Blockwise

In a blockwise CAVI sweep, all K columns of L are updated before F sees any of the changes, and all of F is updated before β is touched. Each update therefore uses stale values from the prior iteration for the other parameters.

In a factor-wise (Gauss-Seidel) sweep, F_k immediately uses the updated L_k from step (b) of the same iteration. β_k then uses the updated L_k and the reused $z^{(-k)}$. When the algorithm advances to factor $k+1$, it uses the freshest values of l_k , f_k , and β_k when computing the residuals and working responses for $k+1$.

This is the true coordinate ascent behavior: each factor update is conditioned on the best currently available estimates of all other parameters. Factor-wise updating is the correct interpretation of V3 Algorithm 1, and it converges in fewer outer iterations than blockwise on typical datasets.

`results/run_modular_simulation.R` is considered a **deprecated benchmark** retained for historical comparison. `fit_modular.R` is the **canonical reusable fitting function** and should be used for all new analyses.

7 Related Files

File	Role
<code>code/fit_modular.R</code>	This module — full CAVI loop
<code>docs/update_beta.qmd</code>	Companion doc for <code>q()</code> update
<code>docs/update_L.qmd</code>	Companion doc for <code>q(L)</code> update
<code>docs/update_F.qmd</code>	Companion doc for <code>q(F)</code> update
<code>docs/update_tau.qmd</code>	Companion doc for <code>q()</code> update
<code>code/update_beta.R</code>	<code>q()</code> module: <code>compute_z_no_k</code> , <code>update_beta_k</code> , <code>update_beta_all</code>
<code>code/update_L.R</code>	<code>q(L)</code> module: <code>compute_R_k</code> , <code>update_L_k</code> , <code>update_L_all</code>
<code>code/update_F.R</code>	<code>q(F)</code> module: <code>update_F_k</code> , <code>update_F_all</code>
<code>code/update_tau.R</code>	<code>q()</code> module: <code>compute_var_term</code> , <code>update_tau</code>
<code>code/Supervised_Bayesian_MF_V2.R</code>	Original monolithic implementation (reference; do not modify)
<code>results/run_modular_simulation.R</code>	Deprecated blockwise benchmark script
<code>derivations/MF_UpdateDerivations/MF_Derivations/CoordinateAlignments (2V1SR).pdf</code>	Coordinate alignment (2V1SR)
<code>derivations/MF_UpdateDerivations/MF_V2_CompatibilityMapping.pdf</code>	Mathon code mapping (17 pages)